

Dense Neural Networks and Backpropagation

Lecture notes for an 80-minute class · Fall 2026 · Project 3, Lecture 1 of 2

How to use these notes

These are written to be taught from by someone who is not the course instructor. They are **deliberately over-specified**: every block has a time budget, the board work is written out in the order it should appear, and the questions students actually ask are answered in the back. You will not need all of it.

Nothing here needs a laptop or a projector except the five figures, which are supplied as PNGs in the `fig/` folder. If the room has no projector, every figure has a **board fallback** noted with it. The derivations are meant for the board, not for slides.

If you are short on time, go straight to "If you fall behind" on the last page — it lists what to cut, in order.

What the students already have

This is a second-year graduate class. By this lecture they have completed two projects, and the lecture leans on both:

- **Project 1 (SVD)**. They have built a data matrix, decomposed it, projected into a low-dimensional space, and classified with nearest centroids. Critically, they have been drilled on the rule that *anything learned from data is a model parameter and must be fit on training rows only* — the SVD basis and the mean included. That rule returns today, unchanged, and you should say so.
- **Project 2 (nonlinear fitting)**. They have written Gauss–Newton and Levenberg–Marquardt from scratch, computed a Jacobian by finite differences and checked it against an analytic one, and seen a fit converge to the wrong answer from a bad starting point. They know what a loss surface is and that it can have more than one minimum.

So today is not their first optimisation and not their first classifier. The honest framing is: **same problem, more parameters, and a solver that scales**. Say that early — it makes neural networks much less mystical, and it is true.

Explicitly NOT in this lecture

Automatic differentiation. Prof. Strey covers reverse-mode AD in its own session. Today, derive backpropagation by hand for a two-layer network and say out loud that the general machinery — which computes these derivatives for any expression graph — is coming separately. If a student asks "is this what autograd does?", the answer is "yes, and you will see exactly how in a later lecture."

Convolutional networks, sequence models, attention. Not today. If asked, the one-line answer is that these are dense layers with weights tied and reused according to a structure in the data, and that everything derived today still applies to them.

Optimisers beyond plain SGD. Mention that Adam exists and is what they will actually use; do not derive it.

At a glance

Start	Min	Block	The one thing they should leave with
0:00	8	Where this fits	Project 2 had 4 parameters and you could see the cost surface. Today: 803, and you cannot.
0:08	12	From a linear model to a layer	One unit. Why the nonlinearity is not optional. Layers collapse without it.

Start	Min	Block	The one thing they should leave with
0:20	8	What a hidden layer buys	Real CTG data: a straight boundary gets 0.816, a 24-unit layer gets 0.868.
0:28	14	Cross-entropy	Softmax, maximum likelihood, and why not squared error.
0:42	18	Backpropagation	The chain rule as bookkeeping. Forward remembers, backward multiplies.
1:00	14	Overfitting, training and test sets	The gap, the turning point, and the leakage rule from Project 1.
1:14	6	Close	What the demo lecture will do. The sting: 0.917 overall, 0.773 on the class that matters.

Before class

- Print the five figures, or have fig/ open in an image viewer. They are referred to as F1–F5.
- Write the dataset facts in a corner of the board and leave them there all hour: 2126 recordings, 21 features, 3 classes (Normal 1655 / Suspect 295 / Pathologic 176). Nearly every number in the lecture refers back to them.
- You will use the board heavily in the backpropagation block. Clear it beforehand and plan to keep the forward-pass equations visible while you derive the backward pass — the whole point is that the backward pass reads the forward pass in reverse.
- The next class is a live-coding demo that builds this network from scratch. Tell them at the start that it is coming, so they know the algebra today has a payoff on Thursday.

0:00 – 0:08 · Where this fits

Open by connecting to what they just finished, not by defining a neural network.

Board — write this first and leave it up

In Project 2 you minimised $S(p) = \sum_i [y(x_i) - f(x_i; p)]^2$ over $p \in \mathbb{R}^4$.

Today we minimise $L(\theta) = \sum_i \ell(y_i, f(x_i; \theta))$ over $\theta \in \mathbb{R}^{803}$.

Then make four points, quickly.

- **It is the same kind of problem.** A parameterised function, a loss that measures disagreement with data, and a search over parameters. Nothing philosophically new has happened.
- **Three things change, and they all follow from the number of parameters.** You cannot plot the cost surface. You cannot afford to build and invert a $J^T J$ of that size every step, so Levenberg–Marquardt is out and first-order methods are in. And you cannot reasonably compute the gradient by finite differences — that would cost one forward pass *per parameter*. Backpropagation is the fix for the third problem, and it is most of today.
- **The function class changes too.** In Project 2 you chose the model — a biexponential, a one-compartment model — because pharmacokinetics told you what shape to expect. Today we have 21 cardiotocography features and no mechanistic model of how they map to fetal distress. So instead of choosing a specific function we choose a flexible *family* and let the data pick a member of it.
- **The price is honesty about generalisation.** A four-parameter mechanistic model that fits 12 subjects is hard to fool. An 803-parameter model that fits 1594 recordings can memorise. That is why half of this lecture is about training and test sets rather than about networks.

The dataset, in one minute

Cardiotocography: simultaneous fetal heart rate and uterine contraction traces, recorded in labour. The 21 features here are *summaries* of those traces computed by an automated system (SisPorto) — baseline rate, accelerations, decelerations, short- and long-term variability, and histogram statistics. Three obstetricians labelled each recording Normal, Suspect, or Pathologic by consensus.

Why this is the right dataset for dense networks: the input is already a fixed-length vector of clinically meaningful numbers with no spatial or temporal structure left in it. There is nothing for a convolution to exploit. A dense network is genuinely the right architecture here, not a stand-in for one.

Worth saying out loud: the classes are 1655 / 295 / 176. Predicting "Normal" every time scores 0.778. Any accuracy you hear today has to be read against that number, and we come back to it at the end.

0:08 – 0:20 · From a linear model to a layer

Build the network up from something they already trust, one step at a time. Resist drawing the big layered graph until the end of this block.

Step 1 — the unit is a weighted sum they have already computed

Board

$$z = w_1 x_1 + w_2 x_2 + \dots + w_p x_p + b = w^T x + b \quad (\text{one number})$$

$$a = \phi(z) \quad (\text{squash it})$$

Prerequisite — this class has NOT covered logistic regression

Do not introduce the unit as "logistic regression you already know". Build it from what they actually have: in Project 1 they projected a recording onto a component, which is exactly $w^T x$. A unit does that, adds an offset, and squashes.

And they already own the second half of the idea: the nearest-centroid boundary between two classes is the perpendicular bisector of the segment joining the centroids — **a straight line**. So "drop ϕ and you get a straight

decision boundary" is a fact about a classifier they built themselves, not an appeal to a model they have not seen.

The word "logistic regression" appears nowhere in the slide deck. Where the linear baseline is quoted it is called the *best straight-boundary model*. If a student asks what it is, say it is the strongest model whose decision boundary is a straight cut, and that they will meet it by name later.

The "neuron" language adds nothing mathematically. What is new is only that we are about to stack these.

Step 2 — a layer is a matrix multiply

Put h of these units side by side, each with its own weights. Stack the weight vectors as columns and the whole layer is one matrix product:

$$\begin{array}{ll} z = Wx + b & W \text{ is } (h \times p), \quad b \text{ is } (h,), \quad z \text{ is } (h,) \\ a = \phi(z) & \text{applied elementwise} \end{array}$$

Stress the shape bookkeeping — it is exactly the discipline from the Project 1 primer, and it is where their code will break on Thursday. In the demo we will batch rows, so it becomes $Z = XW + b$ with X of shape $(N \times p)$ — mention that now so the transpose does not surprise them later.

Step 3 — why the nonlinearity is not optional

This is the most important thirty seconds of the block. Do it on the board as an algebraic argument, not as an assertion.

Board — the collapse argument

Suppose $\phi = \text{identity}$. Then two layers give

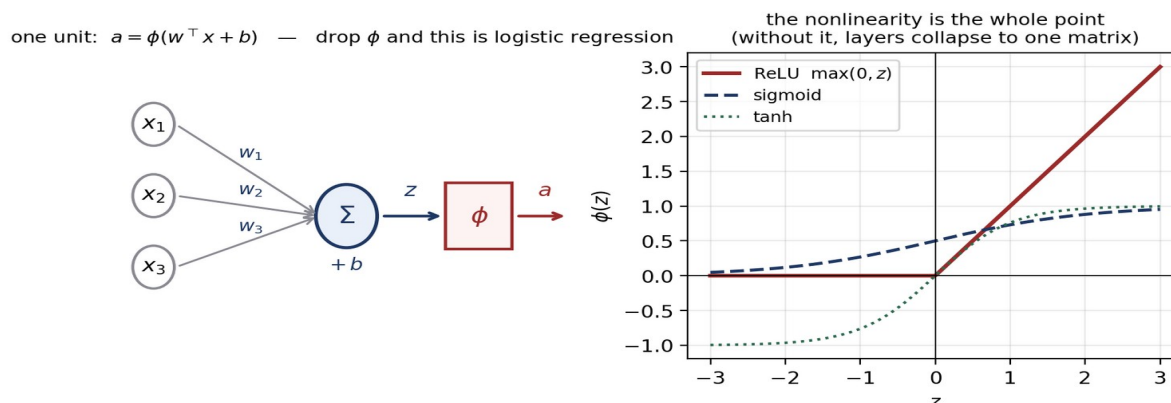
$$a = W_2(W_1x + b_1) + b_2 = (W_2W_1)x + (W_2b_1 + b_2) = \tilde{W}x + \tilde{b}$$

Two layers, one layer. Ten layers, one layer. Depth buys you NOTHING without a nonlinearity between the matrices.

Then introduce the activations themselves (F1, right panel). Keep it brief and practical:

- **ReLU**, $\max(0, z)$. The default. Its derivative is 1 or 0 — nothing else — which is why it does not choke the gradient the way the saturating functions do, and why it is cheap. A unit stuck at negative z for every input contributes no gradient at all; this is the "dying ReLU" and it is worth naming.
- **Sigmoid and tanh**. Historical defaults, still used at outputs. Their derivatives approach zero at both ends, so deep stacks of them train badly — the vanishing-gradient story. Sigmoid survives as a binary output nonlinearity; tanh survives in some recurrent models.

A good aside if you have time: a ReLU network is piecewise linear. It carves the input space into polyhedral regions and is affine on each one. So "curved" decision boundaries like the one in F2 are really many straight pieces — which is visible if you look closely at the figure.



F1 — One unit, and the activation functions. Left: drop ϕ and the unit is a linear model. Right: without ϕ , depth collapses.

Board fallback for F1

Three input circles, arrows into a summation node labelled Σ with $+b$ beneath it, one arrow out labelled z into a box labelled ϕ , one arrow out labelled a . Then sketch ReLU as a hinge at the origin and sigmoid as an S. That is the whole figure.

Step 4 — now draw the network

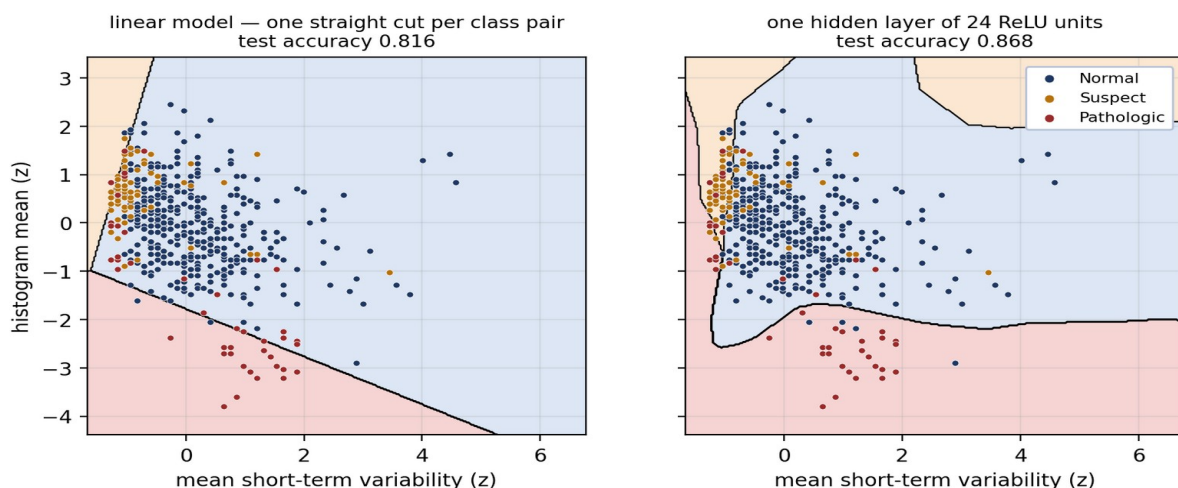
Only now draw input layer $\rightarrow$ hidden layer $\rightarrow$ output layer, and count the parameters out loud for the architecture the demo will use: $21 \rightarrow 32 \rightarrow 3$ gives $21 \cdot 32 + 32 + 32 \cdot 3 + 3 = 803$. Writing that arithmetic on the board is worth more than any diagram: it makes "803 parameters" concrete and it is the number from the opening slide.

Universal approximation — mention and move on. One hidden layer with enough units can approximate any continuous function on a compact set to any accuracy (Cybenko 1989 [3], Hornik 1991 [4]). Say why this is less useful than it sounds: it is an existence result, it says nothing about how many units you need, nothing about whether gradient descent will find those weights, and nothing about generalisation. It justifies the model class; it does not justify anything you will do with it.

0:20 – 0:28 · What the hidden layer actually buys

Short block, but do not skip it — it is the only honest evidence in the lecture that the extra machinery earns its keep, and it is measured on the data they will use on Thursday.

Two real cardiocography features, mean value of short term variability and histogram mean, both standardised. Same data, same train/test split, two models.



F2 — Left: the best straight-boundary model, test accuracy 0.816. Right: one hidden layer of 24 ReLU units, test accuracy 0.868. Test points shown.

What to draw their attention to, in this order:

- **The linear model can only cut with straight lines.** It has to choose one orientation and live with it. Look at the Pathologic cluster at bottom left — to capture it, the straight boundary has to sweep across and swallow a band of Normals on the way.
- **The network wraps around it.** The boundary bends because it is a sum of 24 hinges. Each ReLU unit contributes one fold.
- **The gain is 5.3 points, not 30.** Say this explicitly. On tabular clinical data with informative hand-built features, neural networks beat linear models by a modest margin, not by a revolution. Students arrive expecting magic and it is worth deflating that on real numbers early — it makes them better readers of the literature.

Discussion question (ask it, wait, do not answer immediately)

"Why did I show you a two-feature version when the real dataset has 21?"

The answer you are steering toward: because in two dimensions you can SEE the decision boundary, and in 21

you cannot. This is the same move they made in Project 1 when they plotted the first two components. Visualisation is a two-dimensional privilege; once you go beyond it, you are trusting held-out numbers and nothing else. That is exactly why the train/test discipline in the last block is not bureaucracy.

0:28 – 0:42 · Cross-entropy for classification

They have only ever minimised squared error. The goal of this block is that they can say why squared error is the wrong loss for a classifier, and can write down the gradient that makes the backward pass in the next block almost free.

Softmax: turning three numbers into three probabilities

The network outputs three unconstrained real numbers — the logits. A probability distribution needs them positive and summing to one. Exponentiate, then normalise:

$$p_c = \frac{\exp(z_c)}{\sum_k \exp(z_k)} \quad c = 1 \dots C$$

- Only *differences* of logits matter: adding a constant to every z leaves p unchanged. This is worth stating because it explains the $z - \max(z)$ trick they will write on Thursday to stop $\exp()$ overflowing — it is numerically necessary and mathematically free.
- With $C = 2$ this reduces to the familiar sigmoid of the logit difference. Softmax is not a new idea, it is the multi-class version of what they know.

The loss is maximum likelihood — derive it, do not assert it

This is a three-line derivation and it makes cross-entropy feel inevitable rather than arbitrary. It also directly parallels Project 2, where least squares was the maximum-likelihood estimator under Gaussian noise — say that connection out loud.

Board

The model claims $P(\text{class } c \mid x) = p_c$. Under independence, the likelihood of the data is

$$L = \prod_i p_{\{y_i\}}(x_i)$$

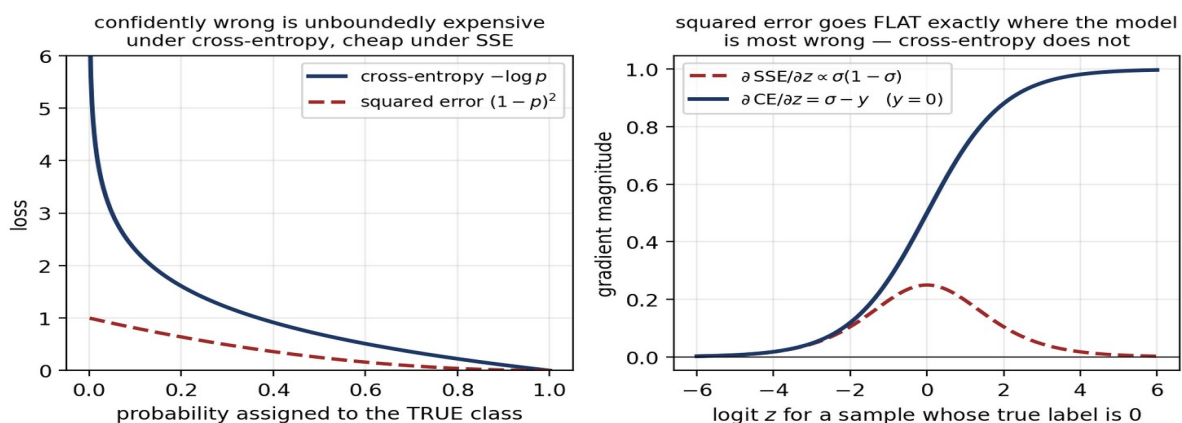
Take $-\log$ and divide by N :

$$\ell = - (1/N) \sum_i \log p_{\{y_i\}}(x_i) \quad \leftarrow \text{cross-entropy}$$

With one-hot y this is $\ell = - (1/N) \sum_i \sum_c y_{ic} \log p_{ic}$.

So minimising cross-entropy is maximising the likelihood of the observed labels. The loss is not a design choice; it is what the probabilistic story forces. Least squares comes from the same argument with Gaussian noise — the only thing that changed is the noise model, because the outcome is now categorical rather than continuous.

Why not squared error?



F3 — Left: the cost of being confidently wrong. Right: the gradient with respect to the logit, for a sample whose true label is 0.

Two arguments, and the second is the one that matters:

- **Cost.** Under cross-entropy, assigning probability 0.01 to the true class costs $-\log(0.01) \approx 4.6$, and the cost grows without bound as $p \rightarrow 0$. Under squared error it costs at most 1. A model that is confidently wrong is barely penalised by squared error, and in a clinical classifier confident and wrong is the worst thing a model can be.

- **Gradient.** This is the decisive one. Look at the right panel. With a sigmoid output and squared error, the derivative with respect to the logit carries a factor of $\sigma(1-\sigma)$, which goes to zero when the model is saturated — that is, precisely when the model is most confidently wrong, it learns most slowly. Cross-entropy cancels that factor exactly. The wronger it is, the harder it is pushed.

The result the whole next block depends on

Board — box this, you will use it in ten minutes

$$\partial \ell / \partial z = p - y$$

Softmax and cross-entropy compose so that the gradient of the loss with respect to the *logits* is just predicted minus actual. No exponentials, no division, no Jacobian of the softmax. One subtraction.

It is worth doing the one-line proof for a single sample if the room is with you: write $\ell = -\log p_y$, substitute the softmax, differentiate with respect to z_c , and the two terms collapse to $p_c - y_c$. If the room is not with you, state it, flag that it is a short exercise, and move on — they will *verify* it numerically on Thursday with a gradient check, which is arguably better than seeing it proved.

Anticipated question: "is accuracy not the thing we care about?"

Yes, and that is exactly why we do not train on it. Accuracy is piecewise constant in the parameters — nudge a weight and either nothing changes or a prediction flips. Its gradient is zero almost everywhere and undefined on the boundaries, so there is nothing for gradient descent to follow. Cross-entropy is a smooth surrogate that agrees with accuracy about which direction is better. Keep accuracy as the reporting metric and never as the training objective.

This distinction comes back in the overfitting block, where the two disagree in an instructive way.

0:42 – 1:00 · Backpropagation

The longest block. The goal is not that they memorise formulas — it is that they see backpropagation as bookkeeping for the chain rule, and understand why it costs about the same as the forward pass.

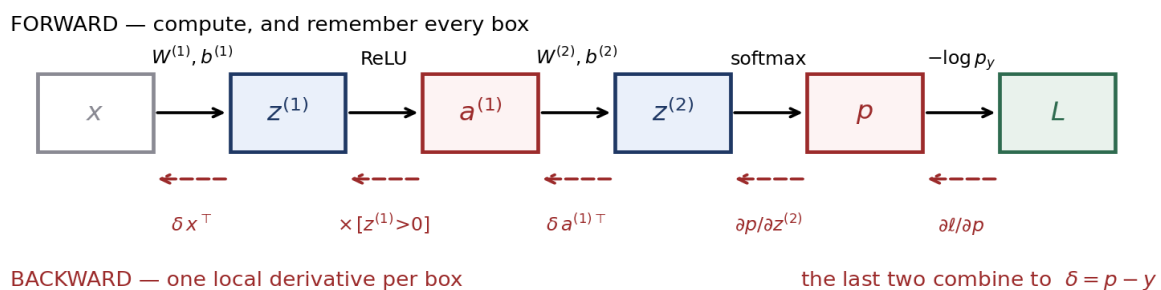
Motivate it by pricing the naive alternative

Start here. In Project 2 they built a Jacobian by finite differences: two extra model evaluations per parameter. For a 4-parameter model that is 8 evaluations and nobody cares. For 803 parameters it is 1606 forward passes **per gradient step**. At a few thousand steps, that is millions of passes through the network to train a small model once. Backpropagation gets the same gradient — all 803 components — for about the cost of two forward passes, and the cost does not grow with the number of parameters.

That ratio is the entire reason deep learning is computationally possible. Say it plainly.

The idea in one sentence

A network is a chain of simple operations. The chain rule says the derivative of the composition is the product of the local derivatives. Backpropagation is nothing more than evaluating that product from the loss end backwards, reusing every partial result instead of recomputing it.



F6 — Forward, then backward, through the same chain. Every backward arrow is one local derivative.

Derive it for the two-layer network

Do this on the board, left to right for the forward pass, then right to left underneath it for the backward pass. Keep both visible at once — the visual alignment is the lesson.

Board — forward (write this row first)

$$z^{(1)} = W^{(1)}x + b^{(1)} \quad a^{(1)} = \text{ReLU}(z^{(1)}) \quad z^{(2)} = W^{(2)}a^{(1)} + b^{(2)} \quad p = \text{softmax}(z^{(2)}) \quad \ell = -\log p_y$$

Board — backward (write this underneath, right to left)

$$\begin{aligned} \delta^{(2)} &= p - y && \leftarrow \text{the boxed result from the last block} \\ \partial \ell / \partial W^{(2)} &= \delta^{(2)} a^{(1)T} && \partial \ell / \partial b^{(2)} = \delta^{(2)} \\ \delta^{(1)} &= (W^{(2)T} \delta^{(2)}) \odot 1[z^{(1)} > 0] && \leftarrow \text{push back, then mask by the ReLU} \\ \partial \ell / \partial W^{(1)} &= \delta^{(1)} x^T && \partial \ell / \partial b^{(1)} = \delta^{(1)} \end{aligned}$$

Three observations to make while it is on the board — these are what they should carry away:

- **Every layer does the same two things.** Turn the incoming δ into a weight gradient by multiplying with the activation it received, and turn it into an outgoing δ by multiplying with its own weights transposed. That is why this generalises to any depth with no new algebra, and why the demo code on Thursday is a loop of four lines.
- **The forward pass has to remember.** $\partial \ell / \partial W^{(2)}$ needs $a^{(1)}$, and the ReLU mask needs $z^{(1)}$. Both are forward-pass quantities. This is why training a network takes far more memory than running one — the activations for the whole batch must be held until the backward pass consumes them. When a student later runs out of GPU memory at a large batch size, this is the reason.

- **The transpose is not a trick.** Forward, W maps a p -vector to an h -vector. The derivative flows the other way, so it needs a map from h back to p — and that is W^T . If a student cannot remember which way round it goes, the shapes decide it for them, every time.

Anticipated question: "why the mask for ReLU?"

Because ReLU is elementwise, so its Jacobian is diagonal, and its derivative is 1 where $z > 0$ and 0 where $z < 0$. Multiplying by a diagonal matrix is the same as multiplying elementwise by its diagonal — hence $\odot$ rather than a matrix product. A unit that was off on the forward pass receives no gradient, which is exactly what "dying ReLU" means, and now they can see why it is permanent: no gradient, no update, still off.

And at exactly $z = 0$? The derivative does not exist. Every library picks 0 or 1 by convention and nothing bad happens, because the event has probability zero in floating point and the loss is still differentiable almost everywhere. This is a good honest moment: the theory is slightly untidy and the practice does not care.

Then: gradient descent, in one minute

```
repeat:
  sample a minibatch of B rows
  forward -> loss
  backward -> gradient g
  theta <- theta - eta * g
```

- **Why minibatches.** A full-batch gradient over 1594 rows costs 1594 forward passes for one step. A 64-row batch gives a noisy estimate of the same gradient for 1/25 of the cost, and you get 25 steps instead of one. Noisy and frequent beats exact and rare. The noise is not only tolerable — it helps the iterate escape bad regions.
- **One epoch = one pass over the training set**, shuffled. Define the word here; the x-axis of every plot in the next block uses it.
- **Learning rate.** The one hyperparameter they cannot ignore. Too large and the loss diverges or oscillates; too small and it crawls. Connect to Project 2: Levenberg–Marquardt adapted its own step size through λ , which is a luxury they no longer have, and mention that Adam is the standard way people buy some of that adaptivity back.

The point to land the block on

Backpropagation is exact. It is not an approximation and it is not a heuristic — it is the chain rule, evaluated in an efficient order. So if your network is not learning, the gradient is almost never the reason, *unless you implemented it yourself*.

And that is checkable, with exactly the technique from Project 2: perturb one parameter by $\pm\epsilon$, recompute the loss, compare the central difference to the analytic gradient. They did this for the Jacobian in Project 2 and got agreement near $1e-5$. On Thursday they will do it for backpropagation and should see agreement near **$1e-7$** . Tell them that number now — it makes Thursday a prediction to be tested rather than a demonstration to be watched.

1:00 – 1:14 · Overfitting, training sets, and test sets

If the lecture runs long, this is the block to protect, not the block to cut. Backpropagation they can read in a book. The discipline in this block is what separates a usable clinical model from a published number that does not replicate.

Start with the rule they already know

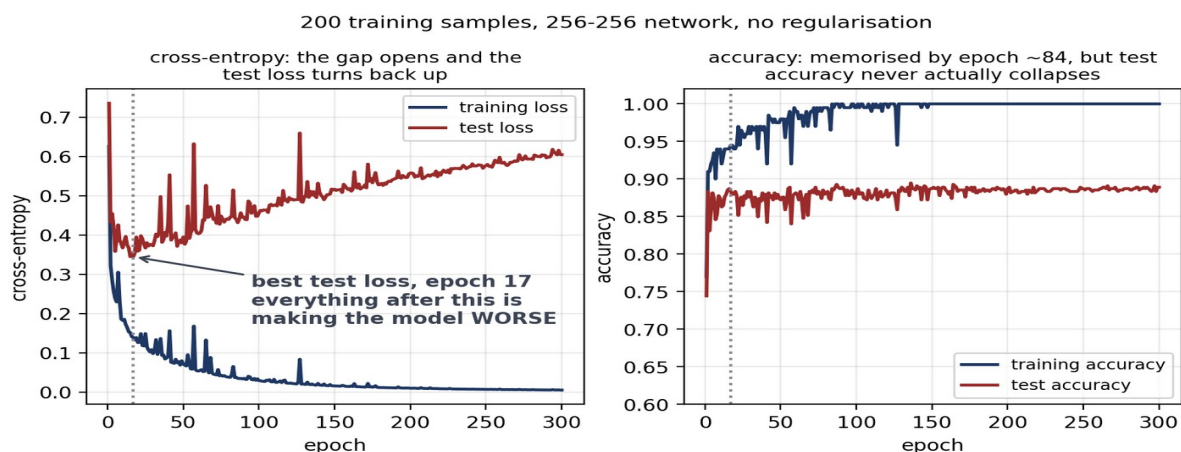
Ask them to state the Project 1 leakage rule back to you. It is: **anything learned from the data is a model parameter, and must be fit on training rows only**. In Project 1 the thing people got wrong was the SVD basis and the mean. Today the list is longer — weights, biases, the feature standardisation, the choice of architecture, the number of epochs you stopped at — but the rule has not changed at all. Feature selection is model fitting. Choosing a hyperparameter by looking at a score is model fitting.

Three splits, three jobs

Split	What it is used for	How many times you may look at it
Training	Fitting weights by gradient descent.	Continuously — that is its job.
Validation	Choosing anything you do not fit by gradient descent: hidden layer size, learning rate, when to stop, how much regularisation.	As often as you like — but every look spends a little of its independence.
Test	Estimating how the finished model will perform on data it has never influenced.	Once. Genuinely once, at the end.

Say the uncomfortable part out loud: almost nobody obeys the last row. If you tune anything against the test set, your reported number is a training score wearing a costume, and the model will do worse in the clinic than on the slide. In this course we use a fixed test split and tune against a validation split — and in Project 1 you split by **subject**, not by row, because two recordings from the same patient are not independent samples.

What overfitting looks like — F4



F4 — 200 training samples, a 256-256 network, 300 epochs, no regularisation. Same run, two views.

Walk them through this figure slowly. It is deliberately a small-sample setting, because that is where overfitting bites — say so, rather than letting them conclude that big networks are always bad.

- **Training loss goes to essentially zero.** It ends at 0.00521, with training accuracy exactly 1.000. The network has memorised all 200 training recordings. That is not a bug and it is not a sign of a good model — with this many parameters it is simply what happens.

- **Test loss reaches its minimum at epoch 17, then turns around.** It climbs from 0.3444 to 0.6043 — 75.4% worse. Every epoch after that point made the model worse at the job it exists to do, while the training curve kept reassuring you.
- **But test accuracy barely moves.** It is 0.883 at the loss minimum and 0.889 at the end — slightly higher. This is the subtle and genuinely useful part of the figure.

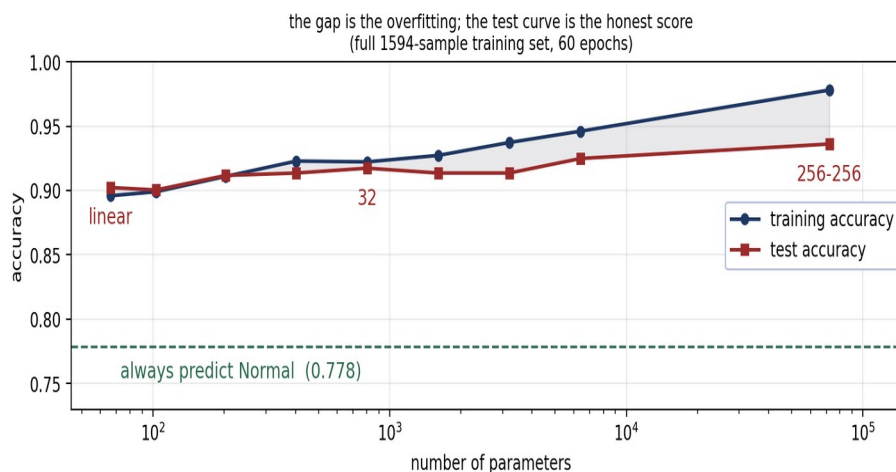
The point worth the whole block — do not skip this

The loss degraded by 75.4% while accuracy did not degrade at all. What rotted was not the *decisions* but the *confidence*. The network kept the same answers and became far more certain about them, including the wrong ones.

Why a biomedical engineer should care: a screening model whose probabilities are miscalibrated is dangerous in a way that accuracy cannot reveal. If it says "92% confident, Normal" when it is right 70% of the time, a clinician acting on that number is being misled — and every accuracy-based validation you ran would have passed. This is why calibration is reported separately in clinical ML, and why "our model is 93% accurate" is an incomplete claim.

A good question to throw to the room: "if accuracy did not get worse, why should I stop early?" Let them argue. The answer is that you are not only making decisions — you are reporting probabilities, and those got worse.

Capacity: the gap is the diagnosis — F5



F5 — Training and test accuracy against parameter count, full training set, 60 epochs. The shaded region is the generalisation gap.

Hidden layers	Parameters	Train	Test	Gap
none (linear)	66	0.896	0.902	-0.006
4	103	0.899	0.900	-0.001
8	203	0.911	0.912	-0.001
16	403	0.923	0.913	+0.009
32	803	0.922	0.917	+0.005
64	1603	0.927	0.913	+0.014
128	3203	0.937	0.913	+0.024
256	6403	0.946	0.925	+0.021
256-256	72195	0.978	0.936	+0.042

Read the table across, not down. Two things are happening at once and students routinely collapse them into one:

- **The gap grows monotonically with capacity.** From essentially zero at the linear model (−0.003, i.e. noise) to +0.046 at 72,195 parameters. That is overfitting, and it is exactly what the classical story predicts.
- **And yet test accuracy also keeps rising**, to its maximum at the largest model. The classical story would have predicted a U — test performance getting worse past some capacity — and on this dataset, at this training set size, that does not happen.

Be honest about this rather than hiding it. Overfitting means the gap, not automatically worse test performance. In the modern over-parameterised regime, test error often keeps improving well past the point of interpolating the training set — the "double descent" phenomenon (Belkin et al. 2019 [11]). Students who have read the classical bias-variance story will find F5 confusing, and they are right to. The operational rule survives regardless: **choose the model by its held-out score, never by the gap and never by the training score.**

What to do about it — name four, do not derive any

- **More data.** Always the best answer and usually unavailable. Notice that F4 used 200 samples and F5 used 1594 — that difference alone is why one shows degradation and the other does not.
- **Early stopping.** Stop at the validation minimum. Nearly free, and it is what the epoch-17 line in F4 is telling you to do. The one thing to emphasise: you choose the stopping epoch on the validation set, not the test set — otherwise you have just fit one more parameter to your test data.
- **L2 regularisation (weight decay).** Add $\lambda\|\theta\|^2$ to the loss; the gradient gains a term $2\lambda\theta$, so it is two characters in the update. In the F4 setting, $\lambda = 1e-2$ holds the final test loss at 0.362 instead of letting it climb to 0.6043. Same architecture, same data, one hyperparameter.
- **Dropout.** Randomly zero a fraction of activations during training only (Srivastava et al. 2014 [9]). Mention it as standard practice; it is not needed for Thursday.

1:14 – 1:20 · Close

Two things: a preview, and a sting. Leave time for the sting — it is the part they should still be thinking about on the walk out.

Preview

Next class is live code. They will write the forward pass, the backward pass and the training loop in NumPy — roughly forty lines — run the gradient check against finite differences, train on all 21 cardiotocography features, reproduce F4 and F5 themselves, and only then call `scikit-learn` and confirm it gets the same answer. Nothing to install beyond NumPy, matplotlib and scikit-learn.

The sting

Put these numbers on the board and let them sit.

```
majority-class baseline (always "Normal")    0.778
best straight-boundary model                0.889
dense network, 21 -> 32 -> 3                0.917

the same network, broken out by class:
recall, Normal      0.969      (414 test recordings)
recall, Suspect     0.716      ( 74 test recordings)
recall, Pathologic  0.773      ( 44 test recordings)
```

Overall accuracy 0.917 sounds good. But the errors are not spread evenly — they are concentrated in the two classes that are the entire reason anyone built the model. Roughly **one Pathologic recording in four is called something else**. A model that is excellent at recognising healthy fetuses and mediocre at recognising distressed ones has the failure mode exactly backwards for its clinical purpose.

And this is fixable, but not for free. Weighting the cross-entropy by inverse class frequency raises Pathologic recall from 0.773 to **0.841** and Suspect from 0.716 to **0.824** — while overall accuracy **falls** from 0.917 to 0.876. There is no setting where you get both. Which trade you want is a clinical judgement about the relative cost of a missed pathology versus a false alarm, and it is not a question machine learning can answer for you. That is the point to end on.

If you have 60 seconds left

Ask: "we measured this model on 44 Pathologic test recordings. How precisely do you actually know its recall on that class?" It got 34 of them right, and an exact 95% confidence interval on 34/44 runs from 0.62 to 0.89 — a span of 27 points. Every number on the board has error bars that nobody printed — which is exactly the lesson from Project 2, arriving in a new place.

If you fall behind

Cut in this order. Each cut buys roughly the time shown and costs roughly the understanding described.

Cut	Buys	What it costs
1. The universal approximation discussion	3 min	Nothing operational. It is a name-check.
2. The maximum-likelihood derivation of cross-entropy	4 min	State the loss and the gradient $p - y$ instead. They lose the sense that the loss is derived rather than chosen — recoverable in a reading.
3. F5 and the capacity table	5 min	They will reproduce it themselves on Thursday. Keep F4.
4. The $\delta^{(4)}$ line of the backward pass	4 min	Do the output layer fully, then say "the same two operations repeat, one layer down". Most of the insight survives.

Do not cut: the collapse argument for why the nonlinearity is needed; the gradient $\partial \ell / \partial z = p - y$; F4 and the loss-versus-accuracy point; and the closing per-class numbers. Those four are the lecture.

Questions students ask

"How do I choose the number of hidden units and layers?"

Empirically, on a validation set — there is no formula, and anyone who gives you one is selling something. Practical starting advice: begin with one hidden layer somewhere between the input and output size, confirm the model can overfit a small subset (if it cannot, something is broken), then add regularisation. The F5 table is exactly this search, run for them.

"Why standardise the features?"

Because the raw features here differ in standard deviation by a factor of about 679,857. Gradient descent takes one shared step size in every direction, so a scale mismatch that large means the step is far too big for one feature and far too small for another. The demo shows the failure live: the same network on unstandardised features collapses to 0.778 — exactly the majority-class baseline, i.e. it learns nothing at all. Fit the scaler on the training rows only — it is a model parameter, same rule as always.

"Are deep networks better than wide ones?"

For data with compositional structure — images, language, audio — depth is dramatically more parameter-efficient, and that is most of why the field moved. For 21 tabular clinical features it matters much less, and gradient-boosted trees are frequently still the strongest model on this kind of data. Worth saying: for tabular problems, a neural network is a reasonable choice, not an obviously correct one.

"Is this what the brain does?"

No, and the name is a historical accident. Real neurons spike, do not share a global clock, and there is no known biological mechanism that transports an error signal backwards through a network the way backpropagation requires. The inspiration was real in 1958 and the resemblance is now mostly nominal.

"Why is the loss noisy from step to step?"

Because each step uses a different minibatch, so the gradient is a random estimate of the full-batch gradient. The jitter is the sampling noise. It is expected, and it tends to help.

"What if I get a different answer when I rerun it?"

You will, unless you fix the seed — weight initialisation and batch shuffling are both random. This is the Project 2 multi-start lesson again: if two runs from different seeds disagree meaningfully, that is information about your model, not noise to be suppressed.

"Could we just use more layers instead of thinking about overfitting?"

That is empirically closer to defensible than it sounds — see the F5 discussion and Belkin et al. [11] — but it does not remove the need for a held-out estimate. More capacity changes where the honest number comes from; it does not change that you need one.

Misconceptions to head off

- **"Overfitting means the test score got worse."** It means the *gap* between training and held-out performance grew. Test performance may get worse, stay flat, or improve. F5 shows a growing gap with improving test accuracy in the same experiment.
- **"The network learns features, so I do not need to think about the data."** These 21 features are the product of decades of obstetric signal processing. A network on raw cardiotocography traces with this much data would do considerably worse. Representation learning needs data volume that clinical studies rarely have.
- **"More epochs is more training, so it must be better."** F4 is the counterexample and the reason early stopping exists.
- **"Softmax outputs are probabilities, so I can trust them as probabilities."** They sum to one, which is not the same as being calibrated. F4 is precisely a model whose probabilities became less trustworthy while its decisions did not change.
- **"Accuracy of 0.92 on a 3-class problem is good."** Against a 0.778 majority baseline with this class imbalance, it needs the per-class breakdown before it means anything at all.

References

- [1] Rumelhart, D. E., Hinton, G. E., & Williams, R. J. (1986). Learning representations by back-propagating errors. *Nature*, 323(6088), 533–536. [doi:10.1038/323533a0](https://doi.org/10.1038/323533a0) — *the paper that made backpropagation widely known.*
- [2] Linnainmaa, S. (1976). Taylor expansion of the accumulated rounding error. *BIT Numerical Mathematics*, 16(2), 146–160. [doi:10.1007/BF01931367](https://doi.org/10.1007/BF01931367) — *reverse-mode differentiation, six years before [1]; the priority claim is his 1970 master's thesis. Relevant to the autodiff lecture.*
- [3] Cybenko, G. (1989). Approximation by superpositions of a sigmoidal function. *Mathematics of Control, Signals and Systems*, 2(4), 303–314. [doi:10.1007/BF02551274](https://doi.org/10.1007/BF02551274)
- [4] Hornik, K. (1991). Approximation capabilities of multilayer feedforward networks. *Neural Networks*, 4(2), 251–257. [doi:10.1016/0893-6080\(91\)90009-T](https://doi.org/10.1016/0893-6080(91)90009-T)
- [5] Bridle, J. S. (1990). Probabilistic interpretation of feedforward classification network outputs, with relationships to statistical pattern recognition. In *Neurocomputing: Algorithms, Architectures and Applications* (NATO ASI Series F, Vol. 68, pp. 227–236). Springer. [doi:10.1007/978-3-642-76153-9_28](https://doi.org/10.1007/978-3-642-76153-9_28) — *the softmax paper.*
- [6] Nair, V., & Hinton, G. E. (2010). Rectified linear units improve restricted Boltzmann machines. In *Proceedings of the 27th International Conference on Machine Learning* (pp. 807–814). Omnipress.
- [7] Glorot, X., Bordes, A., & Bengio, Y. (2011). Deep sparse rectifier neural networks. *Proceedings of the 14th International Conference on Artificial Intelligence and Statistics*, PMLR 15, 315–323. proceedings.mlr.press/v15/glorot11a.html

- [8] He, K., Zhang, X., Ren, S., & Sun, J. (2015). Delving deep into rectifiers: Surpassing human-level performance on ImageNet classification. In *2015 IEEE International Conference on Computer Vision (ICCV)* (pp. 1026–1034). [doi:10.1109/ICCV.2015.123](https://doi.org/10.1109/ICCV.2015.123) — the initialisation scheme the demo code uses.
- [9] Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I., & Salakhutdinov, R. (2014). Dropout: A simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research*, 15, 1929–1958.
- [10] Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press. Chapters 6 (feedforward networks) and 7 (regularisation) match this lecture closely. The complete text is free to read at deeplearningbook.org.
- [11] Belkin, M., Hsu, D., Ma, S., & Mandal, S. (2019). Reconciling modern machine-learning practice and the classical bias–variance trade-off. *Proceedings of the National Academy of Sciences*, 116(32), 15849–15854. [doi:10.1073/pnas.1903070116](https://doi.org/10.1073/pnas.1903070116) — background for the F5 discussion.
- [12] Ayres-de-Campos, D., Bernardes, J., Garrido, A., Marques-de-Sá, J., & Pereira-Leite, L. (2000). SisPorto 2.0: A program for automated analysis of cardiotocograms. *The Journal of Maternal-Fetal Medicine*, 9(5), 311–318. [doi:10.3109/14767050009053454](https://doi.org/10.3109/14767050009053454) — the source of the dataset and of the 21 features.
- [13] Campos, D., & Bernardes, J. (2000). *Cardiotocography* [Dataset]. UCI Machine Learning Repository. [doi:10.24432/C51S4N](https://doi.org/10.24432/C51S4N)